

计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

软件逆向工程

第二讲 汇编语言程序设计基础

主要内容

- 程序设计语言概述
- 寻址方式
- 常见的指令
- 汇编程序结构

程序设计语言概述

□ 程序设计语言是专门为计算机编程所配置的语言。

按照形式的不同可分为：

- 机器语言（Machine Language）
- 汇编语言（Assembler Language）
- 高级语言（High Level Language）

机器语言

- ❑ 是计算机能直接识别的二进制代码组成的指令。
- ❑ 机器语言主要有两个特点：
- ❑ 机器语言与机器密切相关
- ❑ 机器语言设计程序非常困难，但容易实现高性能

汇编语言

- ❑ 汇编语言采用助记符表示机器指令的操作码。
- ❑ 汇编语言不再是直接的二进制代码，而是一种符号语言。比机器语言容易理解和掌握，也容易调试和维护
- ❑ 用汇编语言编写的程序称为汇编源程序，汇编语言源程序要翻译成机器语言程序才可以由计算机执行

高级语言

- ❑ 为了提高编程效率，人们希望通过更熟悉和通用的方式来控制计算机。
- ❑ 高级语言采用更加接近自然语言和惯用的数学表达形式，与计算机硬件结构无关，因而便于使用，便于交流和推广。
- ❑ 高级语言编程效率高，但运行效率低。

汇编语言指令格式

汇编语言指令格式

- 指令的两个要素
 - 要做什么操作？
 - ◆ 助记符
 - 操作对象是什么？
 - ◆ 操作数

指令助记符

MOV

AX, BX

AX

BX

源操作数

目的操作数



将汇编语言转换成机器语言

- ▣ 两步走

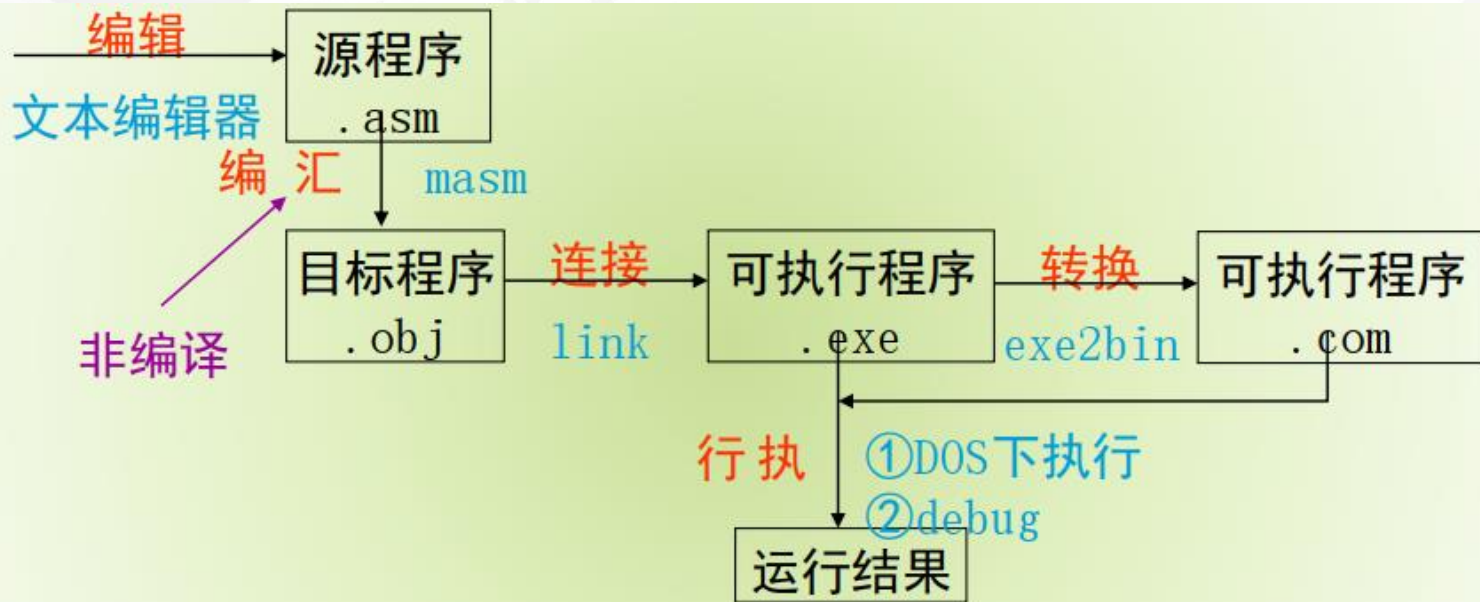
- ▣ 汇编程序

- ▣ 把汇编语言程序翻译成机器语言描述的目标程序的过程。

- ▣ 连接程序

- ▣ 实现多个目标文件及库文件的连接，并完成浮动地址的重定位

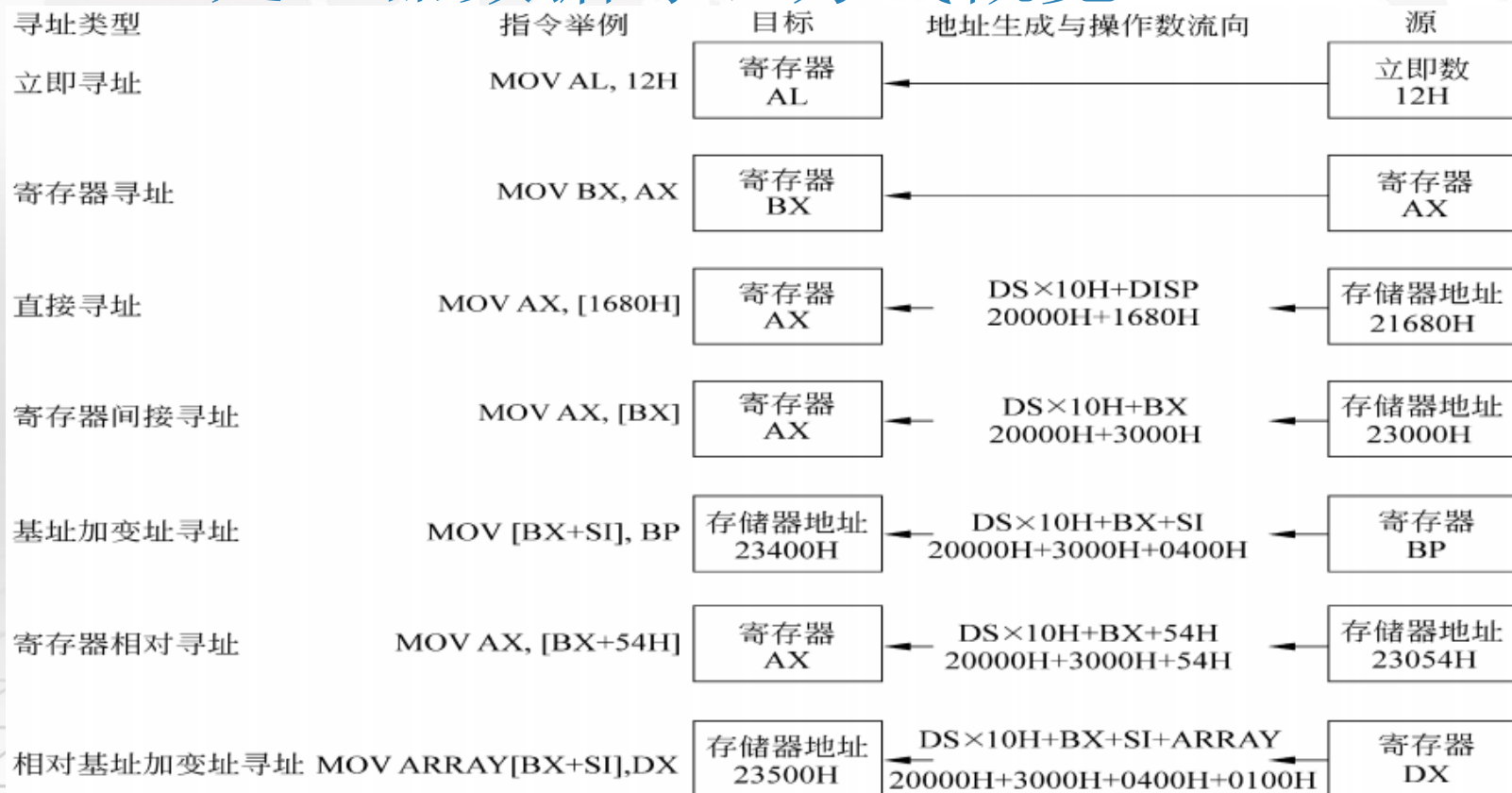
将汇编语言转换成机器语言



指令寻址方式

- 寻址方式就是以什么方式找到操作数

8086处理器数据寻址方式概览



注：BX=3000H, SI=0400H, ARRAY=0100H, DS=2000H

立即寻址

—— 操作数就在指令中提供

汇 编 语 句	长度/位	操 作
MOV AH, 4CH	8	把 4CH 传送到 AH 中
MOV AX, 1234H	16	把 1234H 传送到 AX 中
MOV DI, 0	16	把 0000H 传送到 DI 中
MOV CL, 100	8	把 100(64H)传送到 CL
MOV AL, 'A'	8	把 ASCII 码 A(41H)传送到 AL 中
MOV AX, 'AB'	16	把 ASCII 码 BA* (4241H)传送到 AX 中
MOV CL, 10101101B	8	把二进制数 10101101 传送到 CL 中

立即寻址

- ❑立即寻址时CPU直接从紧跟的指令代码的后续地址单元中取得要操作的数，而不必执行总线周期
- ❑立即数可以是8位，也可以是16位
- ❑立即数寻址主要用来给寄存器赋初值，指令执行速度快

寄存器寻址

□操作数放在CPU寄存器中

汇 编 语 句	长 度 / 位	操 作
MOV AL,BL	8	把 BL 复制到 AL 中
MOV BH,BL	8	把 BL 复制到 BH 中
MOV CX,AX	16	把 AX 复制到 CX 中
MOV SP,BP	16	把 BP 复制到 SP 中
MOV DI,SI	16	把 SI 复制到 DI 中
MOV AX,ES	16	把 ES 复制到 AX 中

寄存器寻址

- ❑ 寄存器寻址可以使用8位或者16位的寄存器，对于8位寄存器，可以是AL、AH、BL、BH、CL、CH、DL、DH
- ❑ 如果源操作数和目的操作数都是寄存器寻址，操作数在CPU内部进行，不需要执行总线周期

直接寻址

□数据在存储器中，有效地址由指令给出

汇 编 语 句	长度/位	操 作
MOV AX,[1680H]*	16	把数据段存储器地址 1680H 和 1681H 两单元的字内容复制到 AX 中
MOV AX,NUMBER	16	把数据段存储器地址 NUMBER 中的字内容复制到 AX 中
MOV TWO,AL	8	把 AL 的字节内容复制到数据段存储单元 TWO 中
MOV ES:[3000H],AX	16	把 AX 的字内容复制到附加数据段存储单元 3000H 中
MOV AX,DATA	16	把数据段存储单元 DATA 的字内容复制到 AX 中

注：* 汇编语言中很少采用绝对偏移地址，通常采用符号地址。

寄存器间接寻址

□数据在存储器中，有效地址由寄存器给出

汇 编 语 句	长度/位	操 作
MOV AL,[BX]	8	把数据段中以 BX 作为有效地址的存储单元的内容(字节)复制到 AL 中
MOV [SI],BL	8	把寄存器 BL 的内容复制到数据段以 SI 作为有效地址的存储单元
MOV CX,[DX]	16	把数据段由 DX 寻址的存储单元的内容(字)复制到 CX 中
MOV [BP],CL*	8	把寄存器 CL 的内容复制到堆栈段以 BP 作为有效地址的存储单元中
MOV [SI],[BX]	—	除数据串操作指令外,不允许由存储器到存储器的传送

注：* 系统把由 BP 寻址的数据默认为在堆栈段中,其他间接寻址方式均默认为数据段。

寄存器间接寻址

- ❑ 寄存器间接寻址允许使用的寄存器BX、BP、SI、DI
- ❑ 书写时，寄存器带方括号[]
- ❑ 当使用BX、DI和SI寻址存储器时，默认使用数据段DS，而使用BP寻址存储器时，默认使用堆栈段SS

基址加变址寻址

□ 数据在存储器中，有效地址由基址寄存器（BX或BP）与变址寄存器（SI或DI）之和给出

汇 编 语 句	长度/位	操 作
MOV CL,[BX+SI]	8	把以 BX+SI 作为有效地址的数据段存储单元的内容(字节)复制到 CL
MOV CX,[BP+DI]	16	把以 BP+DI 作为有效地址的堆栈段存储单元内的内容(字)复制到 CX
MOV [BX+DI],SP	16	把 SP 的内容(字)存入以 BX+DI 作为有效地址的数据段存储单元
MOV [BP+SI],CH	8	把寄存器 CH 的内容(字节)存入以 BP+SI 作为有效地址的堆栈段存储单元

基址加变址寻址

- 在使用基址加变址寻址时，通常用基址寄存器记录存储器数组的起始地址，而变址寄存器记录数组元素的相对位置

寄存器相对寻址

□ 数据在存储器中，有效地址由寄存器与位移量之和给出

汇 编 语 句	长度/位	操 作
MOV CL,[SI+200H]	8	把以 SI+200H 作为有效地址的数据段存储单元的字节内容装入 CL
MOV ARRAY[DI],BL	8	把 BL 中的字节内容存入以 ARRAY+DI 作为有效地址的数据段存储单元
MOV LIST[DI+3],AX	16	把 AX 的字内容存入以 LIST+DI+3 之和作为有效地址的数据段存储单元
MOV AX,ARRAY[BX]	16	把数据段中以 ARRAY+BX 作为有效地址的字内容装入 AX
MOV SI,[AL+12H]	16	把以 AL+12H 作为有效地址的数据段存储单元的字内容装入 SI

相对基址加变址寻址

□ 数据在存储器中，有效地址由基址寄存器、变址寄存器与位移量3个分量之和给出

汇 编 语 句	长度/位	操 作
MOV BL,[BX+SI+100H]	8	把以 $BX+SI+100H$ 作为有效地址的数据段存储单元的字节内容装入 BL
MOV AX,ARRAY[BX+DI]	16	把以 $ARRAY+BX+DI$ 之和作为有效地址的数据段存储单元的字内容装入 AX
MOV LIST[BP+DI],BX	16	把 BX 的字内容存入以 $LIST+BP+DI$ 之和作为有效地址的堆栈段存储单元
MOV AL,LIST[BX+DI]	8	把以 $LIST+BX+DI$ 之和作为有效地址的数据段存储单元的字节内容装入 AL

指令



数据传送类指令

□ 通用数据传送指令

□ MOV、PUSH、POP、XCHG、XLAT

□ 目标地址传送指令

□ LEA、LDS、LES

□ 标志位传送指令

□ LAHF、SAHF、PUSHF、POPF

□ IO数据传送指令

□ IN、OUT

□ MOV AL, BL ; BL中的8位数据送AL

□ MOV ES, DX ; DX中16位数据送ES

□ 注意事项

通用传送指令可传送8位和16位数据

通用传送指令中总是既含源操作数，又含目的操作数

不能在两个内存单元之间直接传送数据

CS和IP寄存器不能作为目的操作数，为什么？

用BX、SI、DI来间接寻址时，默认的段寄存器为DS，而

用BP来间接寻址时，默认的段寄存器为SS

所有的通用传送指令都不改变标志

堆栈操作指令

❑ 推入堆栈指令PUSH

❑ 弹出堆栈指令POP

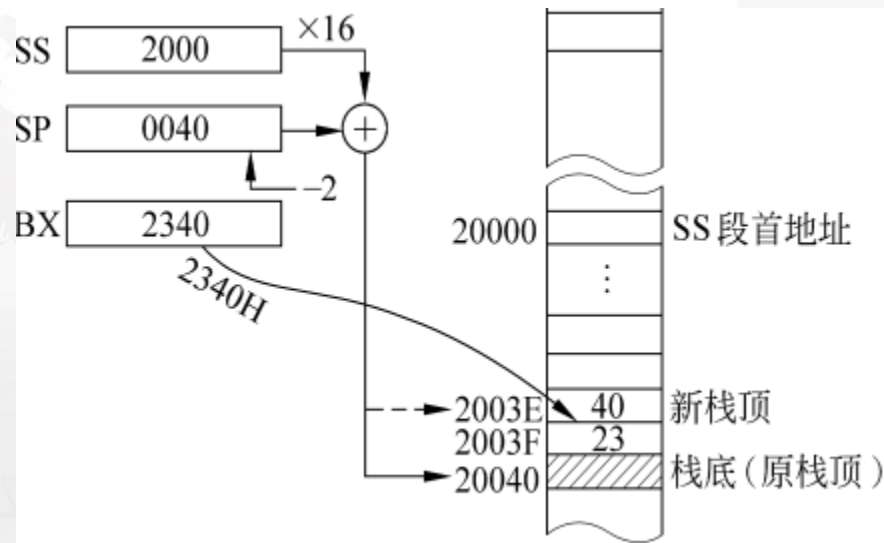
❑ 假设

SS=2000H, SP=0040H,

BX=2340H

执行指令

PUSH BX



算术运算指令

▣ ADD(ADC,INC)、SUB(SUBB,DEC)、MUL(IMUL)、DIV(IDIV)

▣ 所有算术运算均影响标志寄存器的状态标志

CF: 运算最高位产生进位时, CF=1;

OF: 当运算产生溢出时, OF=1;

ZF: 如运算结果为0, 则ZF=1;

SF: 如运算结果为负数, 则SF=1;

PF: 如运算结果中有偶数个1, 则PF=1。

AF: D3位向D4位产生进位, 则AF=1

逻辑运算指令

AND AL, 0FH ; AL中内容和0FH相与, 结果在AL
OR AX, 00F0H ; AX和00F0H相或, 结果在AX
XOR AX, AX ; AX内容本身进行异或, 结果AX清零
NOT AL ; AL中内容求反码, 结果AL中

□TEST和AND执行同样操作, 但TEST指令不送回操作结果, 而仅仅影响标志位。

如:

TEST AX, 8000H ; 如AX的最高位为1, 则ZF=0, 否则ZF=1

移位指令

□ 非循环移位

算术左移指令SAL

逻辑左移指令SHL

算术右移指令SAR

逻辑右移指令SHR

□ 循环移位

循环左移指令ROL

循环右移指令ROR

带进位循环左移指令RCL

带进位循环右移指令RCR

非循环移位指令

□ 非循环移位

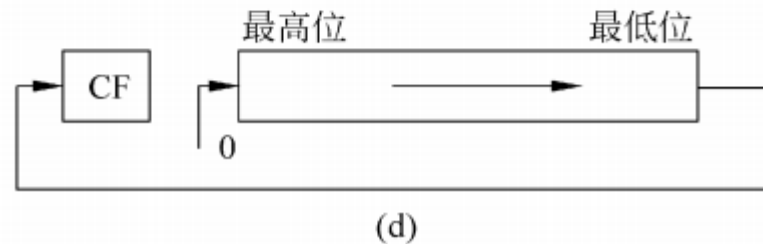
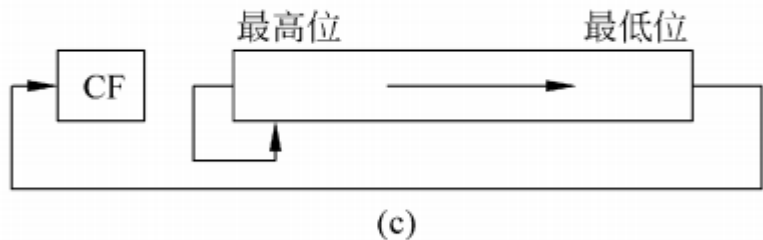
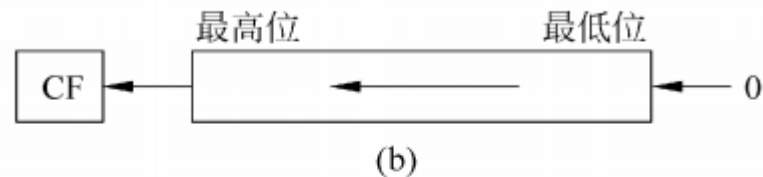
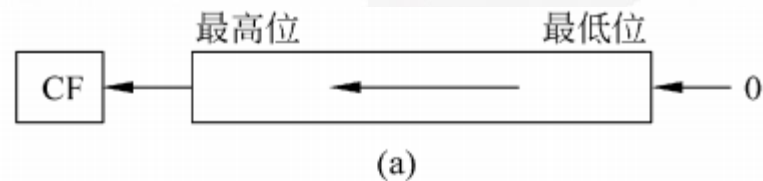
(a) 算术左移指令 SAL

(b) 逻辑左移指令 SHL

(c) 算术右移指令 SAR

(d) 逻辑右移指令 SHR

□ 算术移位是对有符号数进行移位，在移位过程中必须保持符号不变；逻辑移位是对无符号数进行移位，总是用0来填补已空出的位；



循环移位指令

□ 循环移位

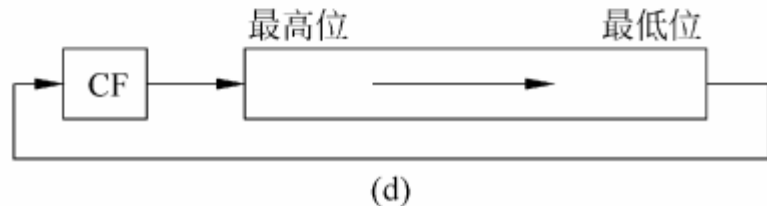
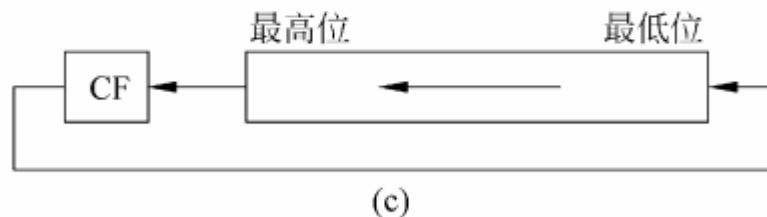
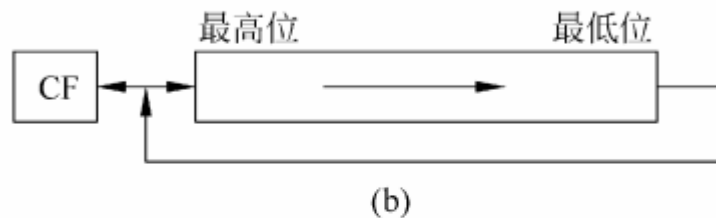
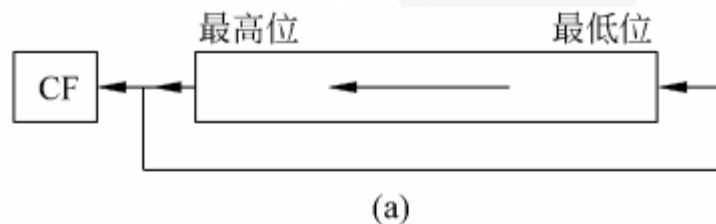
(a) 循环左移指令 ROL

(b) 循环右移指令 ROR

(c) 带进位循环左移指令 RCL

(d) 带进位循环右移指令 RCR

- 循环移位是将操作数首尾相接进行移位



控制转移类指令

□ 转移指令

□ 无条件转移指令 JMP

□ 条件转移指令 J*

JC、JNC、JE/JZ、JNE/JNZ（单标志）

JA/JNBE、JAE/JNB、JB/JNAE、JBE/JNA（无符号数）

JG/JNLE、JGE/JNL、JL/JNGE、JLE/JNG（有符号数）

JO、JNO、JNP/JPO、JP/JPE、JNS、JS（位条件）

□ 循环控制指令

□ LOOP、LOOPE/LOOPZ、LOOPNE/LOOPNZ、JCXZ

□ 过程调用

□ CALL、RET

无条件转移指令JMP

- ❑ 段内直接转移;
JMP 1000H
- ❑ 段内间接转移;
JMP CX
- ❑ 段间直接转移;
JMP 2000H: 0100H
- ❑ 段间间接转移
JMP DWORD PTR [SI]

调用指令CALL

□ 段内直接调用;

◆ CALL 1000H

□ 段内间接调用;

◆ CALL BX

□ 段间直接调用;

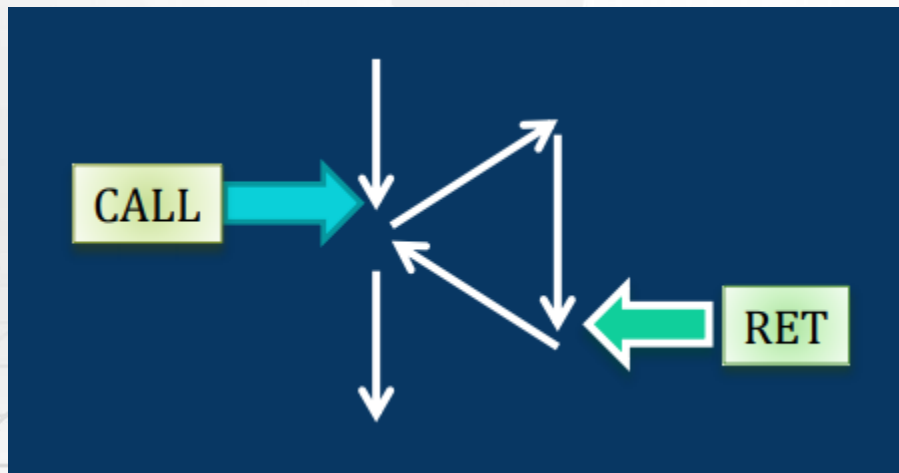
◆ CALL 2000H: 0100H

□ 段间间接调用

◆ CALL DWORD PTR [DI]

调用返回指令RET

▣ 返回高一层的程序，通常与CALL成对使用



条件转移指令

□条件转移指令特点：

1. 都是相对转移形式的 转移范围为-128~+127，好处是指令字节少，执行速度快，需要转移到更远的目标，可以使用中间跳板；
2. 根据比较结果决定是否转移，但注意无符号数和有符号数区分；
3. 大部分转移指令有两种不同的助记符

指令名称			助记符		测试条件
无符号数	高于/不低于也不等于	转移	JA/JNBE	目标标号	CF=0 AND ZF=0
	高于或等于/不低于	转移	JAE/JNB	目标标号	CF=0 OR ZF=1
	低于/不高于也不等于	转移	JB/JNAE	目标标号	CF=1 AND ZF=0
	低于或等于/不高于	转移	JBE/JNA	目标标号	CF=1 OR ZF=1
带符号数	大于/不小于也不等于	转移	JG/JNLE	目标标号	(SF XOR OF) AND ZF=0
	大于或等于/不小于	转移	JGE/JNL	目标标号	SF XOR OF=0 OR ZF=1
	小于/不大于也不等于	转移	JL/JNGE	目标标号	SF XOR OF=1 AND ZF=0
	小于或等于/不大于	转移	JLE/JNG	目标标号	(SF XOR OF) OR ZF=1
单标志位	等于/结果为0	转移	JE/JZ	目标标号	ZF=1
	不等于/结果不为0	转移	JNE/JNZ	目标标号	ZF=0
	有进位/有借位	转移	JC	目标标号	CF=1
	无进位/无借位	转移	JNC	目标标号	CF=0
位条件转移	溢出	转移	JO	目标标号	OF=1
	不溢出	转移	JNO	目标标号	OF=0
	奇偶性为1/偶状态	转移	JP/JPE	目标标号	PF=1
	奇偶性为0/奇状态	转移	JNP/JPO	目标标号	PF=0
	符号位为1	转移	JS	目标标号	SF=1
	符号位为0	转移	JNS	目标标号	SF=0

循环控制指令LOOP

❑ 将CX的内容减1，CX如不为0，则继续循环，为0，则退出循环。

```
MOV    CX, 0100H    ;设置循环次数
KKK:    .....        ;此处为循环体其他代码
        LOOP  KKK     ;CX减1，如不为0则循环
```

有条件循环控制指令

■ LOOPZ/LOOPE

将CX减1，如CX≠0且ZF=1，则继续循环；如CX=0，或ZF=0，则退出循环

■ LOOPNZ/LOOPNE

将CX减1，如CX≠0且ZF=0，则继续循环；如CX=0，或ZF=1，则退出循环

汇编语言基本语法规则



8086汇编语言程序基本结构

```
DATA    SEGMENT
```

```
MESS    DB  'Hello,World!' ,0DH,0AH,24H
```

```
DATA    ENDS
```

```
Stack   SEGMENT PARA STACK
```

```
        DW  256 DUP(?)
```

```
Stack   ENDS
```

```
CODE    SEGMENT
```

```
        ASSUME CS:CODE,DS:DATA
```

```
main:   MOV AX,DATA
```

```
        MOV DS,AX
```

```
        MOV DX,OFFSET MESS
```

```
        MOV AH,9
```

```
        INT 21H
```

```
        MOV AH,4CH
```

```
        INT 21H
```

```
CODE    ENDS
```

```
        END main
```

8086汇编语言程序基本结构

- ❑ 汇编程序一般由若干段组成，每个段都有一个名字，以**SEGMENT**作为段的开始，以**ENDS**作为段的结束，前面都要冠以相同的名字。从段的性质上看，可分为**代码段**、**堆栈段**、**数据段**和**附加段**4种。
- ❑ 汇编程序每一行只有一条语句，且不能超过128个字符。整个源程序必须以**END**语句来结束，**END**后面的标号表示该程序执行时的起始地址。

汇编语言语句的类型

□ 指令性语句

指令性语句是CPU要执行的语句，汇编程序将产生与它一一对应的机器目标代码

如 `MOV DS, AX`

□ 指示性语句(又称为伪指令语句)

指示性语句是只起说明作用而不能执行的语句，它在汇编时只提供一些定义符号、分配存储单元等相关信息

如 `DATA SEGMENT`

指令性语句

□格式

[标号:][前缀]指令助记符[操作数表][;注释]

说明:

标号方便转移指令寻找转移目标地址

前缀配合其他指令 如串操作指令中的REP

助记符即具体指令

操作数表是由逗号隔开的多个操作数

注释是说明文字，提高程序可读性

指示性语句

□格式

[名字]伪操作命令[操作数表][;注释]

X1 DB 40H,50H ;在内存中定义两个字节数据

X2 DW 100,300,500 ;在内存中定义三个字数据

变量数据项

- 组成
- 在除代码段外的段中定义，只由名字组成。
- 属性
- 段属性：变量(定义时)所在段的起始地址；
- 偏移属性：变量与所在段起始地址间字节数；
- 类型属性：表明为它所保留的内存字节数，
DB/DW/DD/DQ/DT对应1/2/4/8/10字节

表达式

□为常数、寄存器名、标号/变量与运算符的组合

□例如：

$12 * 160 + 36 * 2$

$A + B$

$BX + 2$

$BX + SI$

$A[2] + 5$

运算符

- ❑ 算术运算符(+、-、×、/、MOD)
- ❑ 逻辑运算符(AND、OR、XOR、NOT)
- ❑ 关系运算符(EQ、NE、LT、GT、LE、GE)
- ❑ 数值返回运算符(SEG、OFFSET、TYPE、LENGTH、SIZE)
- ❑ 属性运算符(PTR、SHORT、THIS、HIGH、LOW)
- ❑ 注意运算符和指令的区别 `ADD AX, 3+2`

运算符使用示例

MOV AL, 17/5 ; AL=3

MOV AL, VAL AND 01H ; AL=01或00H

MOV AX, DAT GT 0CH ; =0FFFFH

MOV AL, DAT LE 0CH ; =00H

数值返回运算符

	运算符	作用对象	功能
地址回送	SEG	标号或变量	返回其所在段的段基址值
	OFFSET	标号或变量	返回其相对段基址的偏移量
符号特征回送	TYPE	标号或变量	返回用数字表示的类型属性
	LENGTH	变量	若变量用DUP定义，则返回定义元素个数，否则返回1
	SIZE	变量	返回用DUP定义的存储空间(LENGTH*TYPE之积)

TYPE对象种类	TYPE对象类型	TYPE运算结果
变量	DB/DW/DD/DQ/DT	1/2/4/8/10
标号	NEAR/FAR	-1/-2

属性运算符

DATA1 DB 10H, 20H, 30H

DATA2 DW 0123H

DATA3 DW 2345H

.....

MOV AX, WORD PTR DATA1

; (AX) ← 2010H

MOV BL, BYTE PTR DATA2

; (BL) ← 23H

MOV BYTE PTR[SI], 60H

; 字节传送

SUB WORD PTR[BX], 36H

; 字数据相减

JMP FAR PTR SUB_P1

; 段间直接转移

JMP WORD PTR[BX]

; 段内间接转移

MOV AH, HIGH DATA3

; 等价于 MOV AH, 23H

MOV AL, LOW DATA3

; 等价于 MOV AL, 45H

运算符优先级

优先级	运算符	类别
1	()、[]、LENGTH、SIZE	数值回送运算符
2	OFFSET、SEG、TYPE、PTR	属性和数值回送运算符
3	*, /、MOD	算术运算符
4	+, -	算术运算符
5	EQ、NE、LT、GT、LE、GE	关系运算符
6	NOT	逻辑运算符
7	AND	逻辑运算符
8	OR、XOR	逻辑运算符

汇编伪指令

- ❑ 数据定义伪指令
- ❑ 段定义伪指令
- ❑ 程序开始和结束伪指令
- ❑ 表达式赋值伪指令
- ❑ 过程定义伪指令
- ❑ 其他伪指令（地址计数、对准等）

数据定义伪指令

■ 数据定义

□ 格式：[变量名] 类型定义符 参数{[, 参数]}

□ 操作：为变量分配存储空间，并置初值到存储单元

可为DB/DW/DD/DF/DQ/DT，表示每个参数占1/2/4/6/8/10字节存储空间，字符串变量只能用DB类型定义

可为常数、表达式、?表达式、字符串、DUP表达式
字符串用单引号括起来，如 'ABCDE'
DUP表达式格式：n DUP(参数{[, 参数]})

段定义伪指令

- ❑ 格式:
- ❑ 段名 SEGMENT[定位类型][组合类型][类别名]
- ❑ ... ; 语句体
- ❑ 段名 ENDS
- ❑ 操作: 定义逻辑段

段定义伪指令

- ❑ 定位类型，指定当前段起始地址的性质
- ❑ PARA: 起始地址的低4位为0H，即16的倍数（默认选项）
- ❑ PAGE: 起始地址的低8位为00H,即256的倍数
- ❑ BYTE: 可从任意地址开始
- ❑ WORD: 当前段的起始地址为偶地址

段定义伪指令

- ❑ 组合类型，指明程序连接时，段的合并方法
- ❑ PRIVATE：不与其他模块中任何段连接或重叠
- ❑ PUBLIC：不同模块中PUBLIC类型的同名段连接在一起，形成一个单一的段(段长为 Σ)
- ❑ COMMON：不同模块中COMMON类型的同名段重叠在一起，共享相同的存储区(共享存储区长度为max)
- ❑ STACK：功能同PUBLIC，但新段为堆栈段(段长为 Σ)

段定义伪指令

- ❑ 类别名，指明程序连接时，段间次序的确定方法
- ❑ LINK程序将各模块相同‘类别名’的各段依次序连续存放在内存中，但各段相互独立。缺省项为空。

段定义伪指令示例

```
D_SEG SEGMENT  
    NUM1 DB 10 DUP(?)  
D_SEG ENDS
```

```
S_SEG SEGMENT  
    DW 100 DUP(?)  
S_SEG ENDS
```

```
E_SEG SEGMENT  
    STRING DB 'HELLO'  
E_SEG ENDS
```

回顾：

在汇编程序中，从段的性质上看，可分为代码段、堆栈段、数据段和附加段4种。

每个段具体对应到什么性质的段，取决于段寄存器的指向

指定段寄存器伪指令

- ❑ 格式： `ASSUME 段寄存器:段名 [{,段寄存器:段名}]`
- ❑ 操作： 明确段和段寄存器的关系
- ❑ 说明： ①代码段中必须至少有一个ASSUME语句，ASSUME可以出现在源程序中的任何地方；
- ❑ ②对同一段寄存器重复指定时最后一个有效。

指定段寄存器伪指令

```
D_SEG SEGMENT ;数据段
    X DW ?, 4000H, 100 DUP(?)
D_SEG ENDS
E_SEG SEGMENT ;附加段
    STRING DB 'EXAMPLE'
E_SEG ENDS
C_SEG SEGMENT
ASSUME CS:C_SEG, DS:D_SEG, ES:E_SEG
START:
    MOV AX, D_SEG
    MOV DS, AX ;数据段基址→DS
    MOV AX, E_SEG
    MOV ES, AX ;附加段基址→ES
```

必须指定CS

对DS、ES的
指定无效

DS、ES需要在程序
中显式赋值(指定)

指定段寄存器伪指令

■ 堆栈段寄存器SS

- 自动指定: 当有组合类型为STACK的段时, SS自动指向该段

```
S_SEG SEGMENT PARA STACK  
    DW 100 DUP(?)  
S_SEG ENDS
```

思考: 若有多个STACK属性的堆栈段, 结果如何?

- 缺省指定: 当无组合类型为STACK的段时, 无论是否用ASSUME设定, SS自动指向程序区开始 (SP=0000H)

思考: 若未定义堆栈时, 使用堆栈的结果?

- 显式指定: 使用MOV指令, 使SS及SP指向目的处

程序开始和结束伪指令

- ❑ 模块命名伪指令

- ❑ 格式1: `NAME [模块名]`

- ❑ 操作: 给源程序进行模块命名

- ❑ 说明: 若模块名缺省, 则自动用程序名代替。

- ❑ 格式2: `TITLE 标题名`

- ❑ 操作: 给源程序设置标题(模块命名)

- ❑ 说明: 模块名为标题名的前6个字符。

程序开始和结束伪指令

- 程序结束伪指令

- 格式: **END** [地址标号]

- 操作: 表明该模块到此结束

- 说明:

 - ①应用程序只能有一个主模块(地址标号不缺省);

 - ②汇编程序不处理源程序中**END**后的信息。

应用程序与源文件关系

- 应用程序可有多个源程序文件;
- 每个源程序文件为一个模块;
- 所有模块中只能有一个主模块(END后地址标号不省略);
- 每个模块中可有多个段;

表达式赋值伪指令

- 格式：名字 EQU 表达式 或 名字 = 表达式
- 操作：为表达式取一个名字，供以后引用

CONST EQU 100*2 ;定义符号常数

ADDRS EQU [BX+10] ;为地址表达式定义名字

- 说明：
 - 同一源程序中，用EQU赋值的符号不能再次赋值，而用=赋值的符号名可重复赋值
 - 赋值语句不产生目标代码，也不占有存储单元(汇编程序扫描时，用表达式替代符号)，目的是方便更新和提高可读性。

地址计数器伪指令\$

■ 功能：保存当前正在汇编的指令(或数据)的地址

■ 说明：

□ \$为有效地址(16位)；

□ \$可参加构成表达式。

```
ARRAY DW 1, 2, $+4, 3, $+5
```

```
BUFFER DB 1, 2, 3, 4, 5
```

```
COUNT EQU $-BUFFER
```

思考：COUNT的值是什么意思？



ARRAY	01	0074H
	00	
	02	
	00	
	7C	0078H
	00	
	03	
	00	
	81	007CH
	00	

地址计数器伪指令\$

■ 功能：保存当前正在汇编的指令(或数据)的地址

■ 说明：

□ \$为有效地址(16位)；

□ \$可参加构成表达式。

```
ARRAY DW 1, 2, $+4, 3, $+5
```

```
BUFFER DB 1, 2, 3, 4, 5
```

```
COUNT EQU $-BUFFER
```

思考：COUNT的值是什么意思？

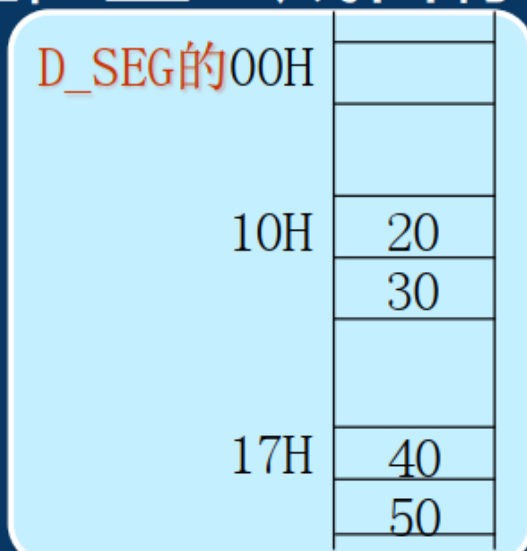


ARRAY	01	0074H
	00	
	02	
	00	
	7C	0078H
	00	
	03	
	00	
	81	007CH
	00	

定位伪指令

- 格式：ORG 表达式
- 操作：将表达式的值送入汇编地址计数器(即命令汇编程序从表达式值位置生成目标代码)

```
D_SEG SEGMENT  
ORG 10H  
X DB 20H, 30H  
ORG $+5  
Y DB 40H, 50H  
D_SEG ENDS
```



顺序程序设计

- 顺序程序是指程序的结构从开始到结尾一直是顺序执行，中途没有分支。
- 试编写程序计算以下表达式：
- $Z=(3X+Y-5)/2$
- 设X、Y的值放在字变量VARX、VARY中，结果存放在
- VARZ中。
- 算法分析：
- 1、乘 2^n 和除 2^n 可以使用算术左移和右移实现
- 2、其它非 2^n 的乘除运算可以用移位和加减组合运算来实现。如 $3X$ 可以分解成 $2X+X$

顺序程序设计

TITLE EQUATION COMPUTE

DATA SEGMENT

VARX DW 15

VARY DW 10

VARZ DW ?

DATA ENDS

STACK1 SEGMENT PARA STACK

 DW 20H DUP (0)

STACK1 ENDS

CODE SEGMENT

 ASSUME CS:CODE,DS:DATA,SS:STACK1

COMP PROC FAR

PUSH DS

MOV AX,0

PUSH AX

MOV AX,DATA

MOV DS,AX

MOV AX,VARX

SHL AX,1 ; $2 * X$

ADD AX, VARX ; $3 * X$

ADD AX, VARY ; $3X + Y$

SUB AX, 5 ; $3 * X + Y - 5$

SAR AX, 1 ; $(3 * X + Y - 5) / 2$

MOV VARZ, AX ; 存结果

RET

COMP ENDP

CODE ENDS

END COMP

实验内容1

- 上网查询DEBUG的使用方法，尝试 u、t、r、d、g等常用命令的使用。

实验内容2：数据段的ARY数组中存放有10个无符号数，
试找出其中最大者送入MAX单元。

算法分析：

依次比较相邻两数的大小，将较大的送入AL中。

每次比较后，较大数存放在AL中，相当于较大的数往下传。

比较一共要做9次。

比较结束后，AL中存放的就是最大数。

DATA SEGMENT

ARY DB 17, 5, 40, 0, 67, 12, 34, 78, 32, 10

MAX DB ?

DATA ENDS

.....

MOV SI, OFFSET ARY ; SI指向ARY的第一个元素

MOV CX, 9 ; CX作次数计数器

MOV AL, [SI] ; 取第一个元素到AL

LOP: INC SI ; SI指向后一个元素

CMP AL, [SI] ; 比较两个数

JAE BIGER ; 前元素 $\geq$ 后元素转移

MOV AL, [SI] ; 取较大数到AL

BIGER: DEC CX ; 减1计数

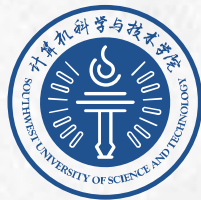
JNZ LOP ; 未比较完转回去, 否则顺序执行

MOV MAX, AL ; 存最大数

.....

实验内容3

- 1、 计算 $1+2+3+\dots+100$
- 思考题，如果要将结果打印显示，该如何实现呢？



计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

谢谢